

Roteiro Prático 08 — Transição para Django e Padrões de Projeto

Objetivos da Aula

Ao final desta aula, o estudante será capaz de:

- Compreender o padrão de arquitetura de software **MVC (Model-View-Controller)** e sua adaptação no Django, o **MTV (Model-Template-View)**;
- Realizar a transição prática de uma aplicação backend baseada em Flask para o framework **Django**;
- Substituir a persistência de dados volátil (lista em memória RAM) pelo **Django ORM** conectado a um banco de dados **SQLite**;
- Desenvolver rotas de API REST em Django que recebem e respondem no formato **JSON** usando **JsonResponse**;
- Habilitar e configurar o painel administrativo nativo do Django (**Django Admin**) para gerenciar os dados da aplicação.

Ferramentas Necessárias

Python, Django, SQLite, JavaScript básico (Fetch API), HTML5/CSS3, terminal, navegador web.

1 Fundamentação Teórica: Padrões de Projeto e MVC vs. MTV

À medida que as aplicações web crescem, organizar o código torna-se um desafio. Para evitar arquivos gigantescos contendo regras de negócio, layouts e lógica de banco de dados misturados (o famoso “código espaguete”), a engenharia de software define os **Padrões de Projeto (Design Patterns)** e **Padrões Arquiteturais**.

O padrão arquitetural mais consagrado para a Web é o **MVC (Model-View-Controller)**:

- **Model (Modelo):** Mapeia a estrutura dos dados e define as regras de negócio. Interage diretamente com o banco de dados.
- **View (Visão):** É a interface de apresentação exibida ao usuário (páginas HTML, telas de celular).
- **Controller (Controlador):** É o cérebro que intercepta as requisições HTTP, solicita os dados necessários ao Model e define qual View apresentará a resposta final.

1.1 O Padrão MTV do Django

O Django adota uma variação desse padrão chamada **MTV (Model-Template-View)**. A lógica é idêntica ao MVC, mudando apenas o papel de alguns componentes:

- **Model:** Igual ao MVC. Define os objetos que serão convertidos em tabelas de banco de dados via ORM.
- **Template (equivalente à View do MVC):** É o arquivo HTML estático ou enriquecido com a sintaxe de template do Django que define a visualização gráfica.
- **View (equivalente ao Controller do MVC):** É a função ou classe Python que recebe a requisição HTTP do usuário, processa a informação e decide se renderizará um Template ou retornará dados puros em JSON.

O fluxo de funcionamento do MTV no Django é ilustrado na Figura 1.

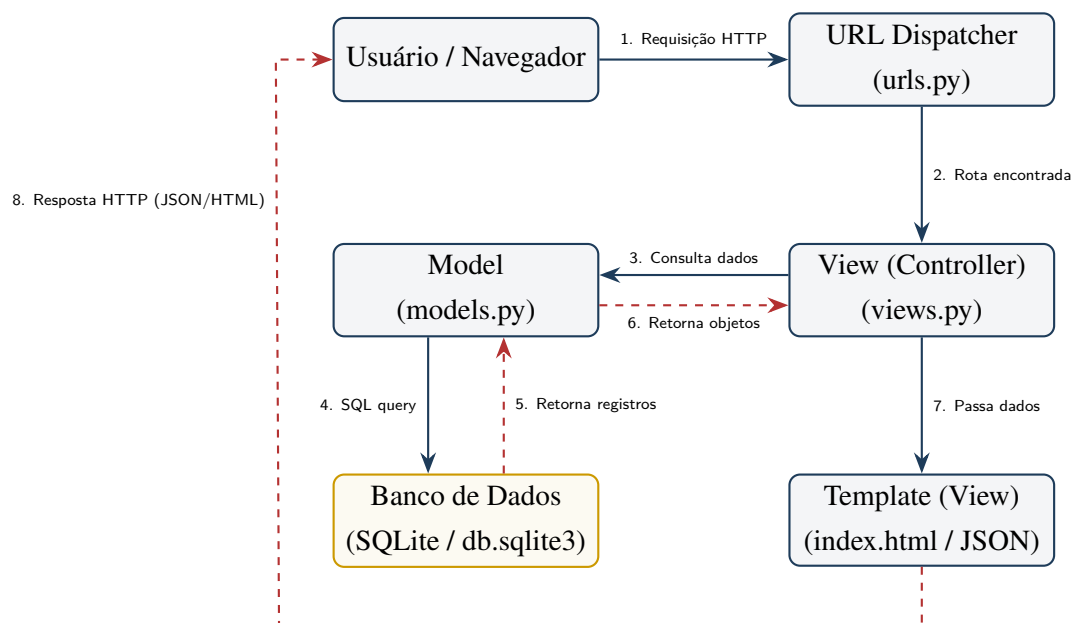


Figura 1: Ciclo de vida de uma requisição HTTP na arquitetura MTV do Django.

2 Comparativo: Flask vs. Django

Para quem já domina o microframework Flask, a tabela a seguir facilita a transição ao mapear os conceitos equivalentes no Django:

Aspecto	Flask (Roteiro 07)	Django (Roteiro 08)
Abordagem	Microframework: Leve, com alta liberdade para estruturar as pastas e escolher bancos de dados externos.	Framework robusto: Estrutura rígida de arquivos pré-configurada, focada em produtividade.
Persistência	Em Memória RAM: Lista estática (<code>alunos = []</code>), que se apaga sempre que o servidor reinicia.	Django ORM: Mapeamento objeto-relacional automático direto em banco SQL local (<code>db.sqlite3</code>).
Mapeamento de Rotas	Decoradores diretamente nas funções de controle (<code>@app.route</code>).	Centralizado no arquivo <code>urls.py</code> da aplicação e do projeto.
Painel de Controle	Não possui (deve ser programado do zero).	Django Admin: Gerado automaticamente a partir dos <code>models</code> registrados.
Retorno JSON	Função <code>jsonify(dados)</code> do Flask.	Classe <code>JsonResponse(dados)</code> de <code>django.http</code> .

3 Mapeamento de Mudanças e Impacto na Aplicação

Para efetuar a transição do Flask para o Django, o nosso código sofrerá reestruturações importantes. A tabela a seguir descreve o que muda e qual o impacto direto dessa alteração na arquitetura e funcionamento do projeto:



Conceito Original (Flask)	Novo Componente (Django)	Mudança Prática e Impacto na Aplicação
<code>aluno.py</code> (Classe Pura)	<code>alunos/models.py</code> (Model)	Herdamos de <code>models.Model</code> . O Django ORM cuida do id autoincrementável e do mapeamento relacional.
Lista RAM (<code>alunos = []</code>)	Banco SQLite (<code>db.sqlite3</code>)	Os dados passam a ser gravados fisicamente em banco de dados, garantindo a persistência das informações mesmo ao reiniciar.
<code>app.py</code> (Arquivo Único)	Views e URLs desacoplados	A lógica de negócio (<code>views.py</code>) fica separada da configuração de rotas (<code>urls.py</code>), facilitando a modularidade.
Decorador <code>@app.route()</code>	Roteamento em <code>urls.py</code>	Todas as URLs do sistema são mapeadas de forma centralizada e organizada em listas de caminhos (<code>path</code>).
Função <code>jsonify()</code>	Classe <code>JsonResponse()</code>	Retorno nativo estruturado em JSON contendo os cabeçalhos corretos gerenciados pelo próprio framework.

3.1 Fluxo Comparativo de Arquitetura (Croqui de Transição)

Abaixo apresentamos o croqui de transição estrutural. Ele ilustra o caminho da requisição HTTP e a persistência dos dados entre a abordagem em Flask desenvolvida anteriormente e a nova arquitetura baseada no Django:

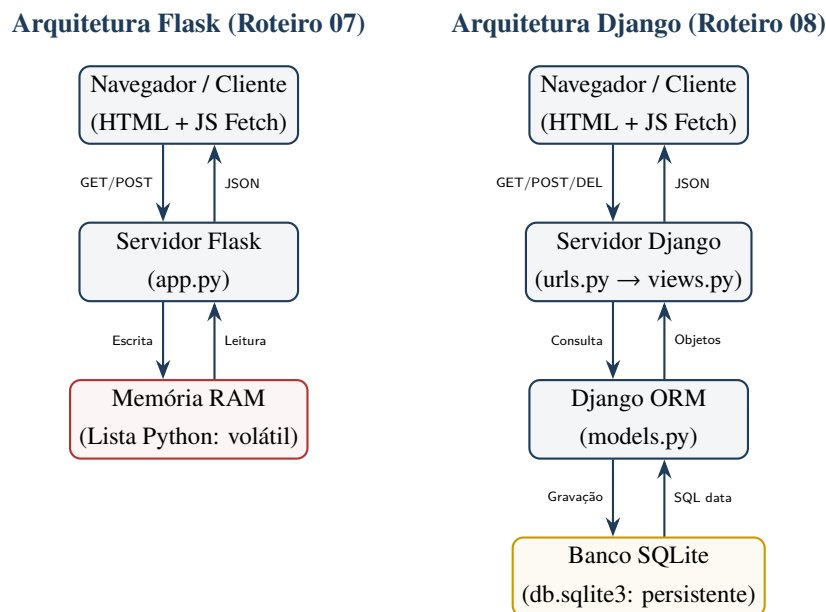


Figura 2: Mapeamento dos fluxos de dados e croqui de arquitetura: Flask vs. Django.

4 Roteiro Prático: Criando a Aplicação Escola no Django

Vamos recriar o backend da nossa aplicação de alunos usando a arquitetura modular do Django, preservando a interface dinâmica SPA que construímos na aula anterior.

Parte 0 – Criando o Projeto e App no Django

Por que fazemos isso? O Django exige que organizemos o código em um contêiner global (o **Projeto**) e em módulos independentes de funcionalidades (os **Apps**). Aqui, vamos instalar o framework e inicializar a estrutura básica do projeto `escola_project` e o app `alunos`.

Abra o terminal com seu ambiente virtual `venv` ativado e execute os seguintes comandos:

```
# 1. Instale o Django na venv
pip install django

# 2. Crie o projeto "escola_project" na pasta atual
django-admin startproject escola_project .

# 3. Crie o app "alunos"
django-admin startapp alunos
```

A estrutura de diretórios gerada automaticamente na raiz do seu projeto deve ser semelhante a esta:

```
roteiro08-django-padroes/
|-- escola_project/
|   |-- __init__.py
|   |-- asgi.py
|   |-- settings.py      # Configurações globais
|   |-- urls.py          # Rotas centrais
|   |-- wsgi.py
|-- alunos/
|   |-- migrations/      # Histórico de migrações do banco
|   |-- __init__.py
|   |-- admin.py         # Configuração do Django Admin
|   |-- apps.py
|   |-- models.py        # Modelos (Estrutura de dados)
|   |-- tests.py
|   |-- views.py         # Lógica das rotas (Controllers)
|-- manage.py            # Utilitário de linha de comando
```

Parte 1 – Registrando a App e Configurações

Por que fazemos isso? O Django precisa saber quais módulos (apps) estão ativos no projeto para carregar seus modelos e rotas. Também configuramos o idioma e o fuso horário para que o banco e o sistema operem no padrão local de forma nativa.

Abra o arquivo `escola_project/settings.py` e registre nosso app, além de configurar as datas e idioma da aplicação:

1. Na lista `INSTALLED_APPS`, adicione o app `'alunos'`:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
  
    # Nosso app registrado  
    'alunos',  
]
```

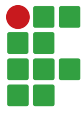
2. Altere o idioma e fuso horário no final do arquivo:

```
LANGUAGE_CODE = 'pt-br'  
TIME_ZONE = 'America/Sao_Paulo'
```

Parte 2 – Definindo o Modelo com Django ORM (`alunos/models.py`)

Por que fazemos isso? A camada Model define a estrutura lógica dos dados. O Django ORM permite que definamos tabelas de banco de dados usando classes Python simples. O framework se encarrega de criar e gerenciar a chave primária (id) automaticamente de forma autoincremental.

Substitua o conteúdo de `alunos/models.py` pelo código Python abaixo. Note que o ID é gerado automaticamente pelo Django, não exigindo controle manual:



```
from django.db import models

class Aluno(models.Model):
    nome = models.CharField(max_length=100, verbose_name="Nome Completo")
    idade = models.IntegerField(verbose_name="Idade")
    nota = models.FloatField(default=0.0, verbose_name="Média")

    def aprovado(self):
        return self.nota >= 7.0

    def to_dict(self):
        return {
            "id": self.id,
            "nome": self.nome,
            "idade": self.idade,
            "media": self.nota,
            "aprovado": self.aprovado(),
        }

    def __str__(self):
        return f"{self.nome} ({self.idade} anos)"
```


Parte 3 – Criando e Aplicando Migrações

Por que fazemos isso? O Django não altera as tabelas do banco de dados automaticamente ao salvarmos o arquivo `models.py`. Precisamos primeiro gerar os planos de migração (`makemigrations`) e depois executá-los (`migrate`) para que o banco físico SQLite seja sincronizado.

Sempre que alteramos a estrutura dos nossos modelos, precisamos rodar as migrações para refletir as mudanças nas tabelas físicas do banco SQLite:

```
# 1. Gera os roteiros de migração na pasta migrations/  
python manage.py makemigrations  
  
# 2. Executa fisicamente e cria as tabelas no SQLite (db.sqlite3)  
python manage.py migrate
```

Parte 4 – Criando as Views da API REST (alunos/views.py)

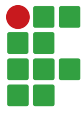
Por que fazemos isso? A View (Controller) gerencia as requisições HTTP enviadas pelo usuário. Como nosso frontend é uma aplicação de página única (SPA), nossas views não devem retornar páginas HTML inteiras em cada requisição; em vez disso, retornam dados estruturados em JSON (`JsonResponse`) para que o JavaScript os renderize na tela.

Abra `alunos/views.py` e insira as funções de visualização. Como faremos requisições assíncronas do JavaScript usando JSON, utilizaremos a classe `JsonResponse` e o decorador `@csrf_exempt` para simplificar a autenticação de rotas POST/DELETE em desenvolvimento local.

Primeiro, insira os *imports* necessários e a rota que carrega a tela inicial do sistema:

```
import json  
from django.http import JsonResponse  
from django.views.decorators.csrf import csrf_exempt  
from django.shortcuts import render  
from .models import Aluno  
  
# Tela inicial (Renderiza index.html)  
def pagina_inicial(request):  
    return render(request, 'index.html')
```

Em seguida, adicione as rotas de API responsáveis por listar, cadastrar e deletar os alunos:



```
@csrf_exempt
def api_alunos(request):
    # GET: Listar todos os alunos
    if request.method == 'GET':
        lista_alunos = Aluno.objects.all()
        dados = [aluno.to_dict() for aluno in lista_alunos]
        return JsonResponse(dados, safe=False, status=200)

    # POST: Criar novo aluno
    elif request.method == 'POST':
        payload = json.loads(request.body)
        nome = payload.get('nome')
        idade = payload.get('idade')
        media = payload.get('media', 8.5) # Valor padrão

        if not nome or not isinstance(idade, int):
            return JsonResponse({"erro": "Dados inválidos."}, status=400)

        novo = Aluno.objects.create(nome=nome, idade=idade, nota=media)
        return JsonResponse(novo.to_dict(), status=201)

@csrf_exempt
def api_alunos_detalhe(request, id):
    # DELETE: Remover aluno
    if request.method == 'DELETE':
        try:
            aluno = Aluno.objects.get(id=id)
            aluno.delete()
            return JsonResponse(
                {"mensagem": "Removido com sucesso"}, status=200
            )
        except Aluno.DoesNotExist:
            return JsonResponse({"erro": "Nao encontrado"}, status=404)
```

Parte 5 – Mapeando as URLs da Aplicação

Por que fazemos isso? O Django usa o arquivo `urls.py` (URL Dispatcher) para mapear os caminhos digitados no navegador e associá-los às funções controladoras corretas do arquivo `views.py`. Criamos arquivos locais no app e os incluímos no roteador central do projeto para garantir a modularidade. Configuraremos o sistema de roteamento criando as URLs do aplicativo e incluindo-as nas rotas centrais do projeto.

1. Crie o arquivo `alunos/urls.py` com o seguinte conteúdo:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.pagina_inicial, name='pagina_inicial'),
    path('api/alunos', views.api_alunos, name='api_alunos'),
    path('api/alunos/<int:id>',
         views.api_alunos_detalhe,
         name='api_alunos_detalhe'),
]
```

2. Abra o arquivo central de rotas `escola_project/urls.py` e configure o include:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('alunos.urls')),
]
```

Parte 6 – Integrando a Interface SPA Estática (`index.html`)

Por que fazemos isso? O Django organiza recursos de visualização de forma isolada. Colocamos arquivos HTML na pasta `templates/` e arquivos auxiliares (como CSS e JavaScript) na pasta `static/`, ensinando o framework a localizá-los e servi-los de forma otimizada com tags como `{% load static %}`.

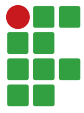
O Django possui pastas específicas para arquivos HTML (`templates/`) e arquivos de recursos (`static/`). Crie essa hierarquia de pastas no seu terminal:

```
mkdir -p alunos/templates  
mkdir -p alunos/static
```

1. Crie o arquivo `alunos/templates/index.html` e cole o código adaptado. A primeira parte contém a estrutura HTML básica e o formulário de cadastro:

```
{% load static %}  
<!DOCTYPE html>  
<html lang="pt-br">  
<head>  
    <meta charset="UTF-8">  
    <title>Controle de Alunos (Django)</title>  
    <link rel="stylesheet" href="{% static 'style.css' %}">  
</head>  
<body>  
    <h1>Controle de Alunos</h1>  
    <form id="formulario-cadastro">  
        <input type="text" id="nome" placeholder="Nome do aluno" required>  
        <input type="number" id="idade" placeholder="Idade do aluno" required>  
        <button type="submit">Cadastrar via API</button>  
    </form>  
    <hr>  
    <h2>Alunos Cadastrados</h2>  
    <ul id="lista-alunos"></ul>
```

A segunda parte inicia a tag `<script>` com as variáveis globais, a função de carregamento assíncrono e a renderização dos alunos na tela:

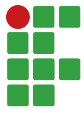


```
<script>
  const API_URL = '/api/alunos';
  const listaAlunos = document.getElementById('lista-alunos');
  const formulario = document.getElementById('formulario-cadastro');
  const inputNome = document.getElementById('nome');
  const inputIdade = document.getElementById('idade');

  function carregarAlunos() {
    fetch(API_URL)
      .then(res => res.json())
      .then(alunos => {
        listaAlunos.innerHTML = '';
        alunos.forEach(aluno => desenharAlunoNaTela(aluno));
      });
  }

  function desenharAlunoNaTela(aluno) {
    const li = document.createElement('li');
    li.id = `aluno-${aluno.id}`;
    li.innerHTML = `
      <span><strong>${aluno.nome}</strong> (${aluno.idade} anos)
      - Média: ${aluno.media}
      [${aluno.aprovado ? 'Aprovado' : 'Reprovado'}]</span>
      <button class="btn-deletar"
        onclick="deletarAluno(${aluno.id})">Deletar</button>
    `;
    listaAlunos.appendChild(li);
  }
}
```

Por fim, a terceira parte contém os eventos de submissão do formulário, a função para deletar aluno e o encerramento da página:



```
formulario.addEventListener('submit', function (event) {
    event.preventDefault();
    const dados = {
        nome: inputNome.value,
        idade: parseInt(inputIdade.value),
        media: 8.5
    };

    fetch(API_URL, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(dados)
    })
        .then(res => res.json())
        .then(aluno => {
            desenharAlunoNaTela(aluno);
            formulario.reset();
        });
});

function deletarAluno(id) {
    if (!confirm('Deseja realmente deletar?')) return;
    fetch(`${API_URL}/${id}`, { method: 'DELETE' })
        .then(res => {
            if (res.ok) {
                const el = document.getElementById(`aluno-${id}`);
                if (el) el.remove();
            }
        });
}

window.addEventListener('DOMContentLoaded', carregarAlunos);
</script>
</body>
</html>
```



2. Crie o arquivo `alunos/static/style.css` com regras CSS premium equivalentes para renderização da página.



Parte 7 – Habilitando o Painel Django Admin

Por que fazemos isso? Um dos diferenciais do Django é fornecer uma interface administrativa robusta pré-construída para que desenvolvedores gerenciem dados em tabelas do banco de dados graficamente, eliminando a necessidade de construir painéis de controle administrativos do zero.

O Django admin permite gerenciar os registros do banco de dados graficamente.

1. Registre o modelo Aluno no arquivo `alunos/admin.py`:

```
from django.contrib import admin
from .models import Aluno

admin.site.register(Aluno)
```

2. No terminal, gere o usuário administrativo global (**superuser**):

```
python manage.py createsuperuser
```

Insira o nome de usuário (ex: admin), um e-mail de contato e defina uma senha.

5 Testando a Aplicação e Validando Persistência

Vamos inicializar o servidor de desenvolvimento e testar os recursos implementados:

1. Execute o servidor Django:

```
python manage.py runserver
```

2. Acesse `http://127.0.0.1:8000/` no seu navegador e cadastre alguns alunos.
3. Abra as Ferramentas do Desenvolvedor (**F12**) no navegador e selecione a aba **Rede (Network)** sob o filtro **Fetch/XHR** para observar as requisições assíncronas assinaladas em segundo plano ao inserir ou remover registros.
4. **Verificação de Persistência no SQLite:** Desligue o servidor no terminal (**Ctrl + C**) e inicie-o novamente. Atualize a página do navegador. Perceba que, ao contrário do Flask CRUD em memória RAM do Roteiro 07, os dados permanecem salvos pois estão armazenados em definitivo no banco local `db.sqlite3`.



5. Acesse <http://127.0.0.1:8000/admin> para logar com o superusuário cadastrado e gerenciar seus modelos de alunos diretamente pela interface administrativa oficial do Django.